Submission Worksheet

CLICK TO GRADE

https://learn.ethereallab.app/assignment/IT114-005-F2024/it114-module-5-project-milestone-1/grade/bna24

Course: IT114-005-F2024
Assigment: [IT114] Module 5 Project Milestone 1
Student: Bakr A. (bna24)

Submissions:

Submission Selection

1 Submission [submitted] 10/21/2024 1:01:20 AM

Instructions

COLLAPSE

Overview Video: https://youtu.be/A2yDMS9TS1o

1. Create a new branch called Milestone1
2. At the root of your repository create a folder called Project if one doesn't exist yet
    1. You will be updating this folder with new code as you do milestones
    2. You won't be creating separate folders for milestones; milestones are just branches
3. Copy in the code from Sockets Part 5 into the Project folder (just the files)
    2. https://github.com/MattToegel/IT114/tree/M24-Sockets-Part5
4. Fix the package references at the top of each file (these are the only edits you should do at this point)
5. Git add/commit the baseline and push it to github
6. Create a pull request from Milestone1 to main (don't complete/merge it yet, just have it in open status)
7. Ensure the sample is working and fill in the below deliverables 1. Note: Don't forget the client commands are /name and /connect
8. Generate the output file once done and add it to your local repository
9. Git add/commit/push all changes
10. Complete the pull request merge from the step in the beginning
11. Locally checkout main
12. git pull origin main

Branch name: Milestone1

Group

100%

Group: Start Up
Tasks: 2
Points: 3

COLLAPSE

Task

100%

Group: Start Up
Task #1: Start Up
Weight: ~50%
Points: ~1.50

COLLAPSE

Details:

Important: Code screenshots should be fairly concise (try to show only the sections of code relevant to the question)

Capturing all possible code (i.e., including a lot of irrelevant code) can lead to a reduced grade.

Columns: 4

Sub-Task

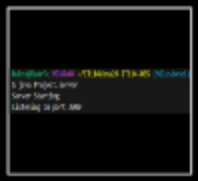
100%

Group: Start Up Task #1: Start Up Sub Task #1: Show the Server

Task Screenshots

Gallery Style: 2 Columns

4 2 1



Server Started and Listening on port 3000

Caption(s) (required) ✓

Caption Hint:

Describe/highlight what's being shown

Sub-Task

100%

Group: Start Up Task #1: Start Up Sub Task #2: Show the Server

Task Screenshots

Gallery Style: 2 Columns

4 2 1

Server code that listens to connection

Rest of the code

Caption(s) (required) ✓

Caption Hint:

Describe/highlight what's being shown (ucid/date must be present)

Sub-Task

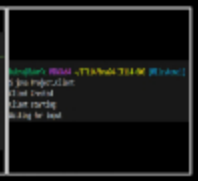
100%

Group: Start Up Task #1: Start Up Sub Task #3: Show the Client

Task Screenshots

Gallery Style: 2 Columns

4 2 1



Client Starting via command line

Caption(s) (required) ✓

Caption Hint:

Describe/highlight what's being shown

Sub-Task

100%

Group: Start Up Task #1: Start Up Sub Task #4: Show the Client

Task Screenshots

Gallery Style: 2 Columns

4 2 1

This starts the client

Listens for keyboard input

Caption(s) (required) ✓

Caption Hint:

Describe/highlight what's being shown (ucid/date must be present)

Task

Response

Prompt

Briefly explain the code related to starting up and waiting for connections

Response:

The start(int port) method begins the server and sets the port and creates a default 'LOBBY' room. It then waits in a loop until client connections arrive using a ServerSocket. To handle multiple clients at once, a ServerThread is created when a client connects. The server shut down cleanly if there's an error.

Response

Prompt

Briefly explain the code/logic/flow leading up to and including waiting for user input

Response:

The start() method starts the client by initializing the client and then starts a separate thread to process user input with a Scanner. It is in continuous loop reading input from the console. Then each line of input is processed to see if it's a particular command, like connecting to a server. The client tries to connect if the command is valid. If the client is connected, the message is sent to the server for regular messages.

End of Task 1

Task

100%

Group: Start Up
Task #2: Connecting
Weight: ~50%
Points: ~1.50

COLLAPSE

Details:

Important: Code screenshots should be fairly concise (try to show only the sections of code relevant to the question)

Capturing all possible code (i.e., including a lot of irrelevant code) can lead to a reduced grade.

Columns: 4

Sub-Task

100%

Group: Start Up
Task #2: Connecting
Sub Task #1: Show 3

Sub-Task

100%

Group: Start Up
Task #2: Connecting
Sub Task #2: Show

Clients

Task Screenshots

Gallery Style: 2 Columns

4 2 1

3 Clients: Bakr, User2, User3

Caption(s) (required) ✓

Caption Hint:

Describe/highlight what's being shown

the code

Task Screenshots

Gallery Style: 2 Columns

4 2 1

establish a connection

checks if the input text is a valid connection command

processes user input commands

Caption(s) (required) ✓

Caption Hint:

Describe/highlight what's being shown (ucid/date must be present)

Task Response Prompt

Briefly explain the code/logic/flow

Response:

the connect method of the code creates a Socket and sets up ObjectOutputStream and ObjectInputStream for communication between the client and server. The isConnection(String text) function is just a check to see if the user input is an IP address or "localhost" with a port number. The

processClientCommand(String text) method processes user commands, determining whether the input is a valid connection command. Then it checks to see if the client name is valid and if so, it starts the connection and sends the client's name. It also handles /quit for disconnecting and /name for setting the clients' name.

End of Task 2

End of Group: Start Up
Task Status: 2/2

Group

Group: Communication
Tasks: 2
Points: 3

COLLAPSE

Task

Group: Communication
Task #1: Communication
Weight: ~50%
Points: ~1.50

COLLAPSE

Details:
Important: Code screenshots should be fairly concise (try to show only the sections of code relevant to the question)

Capturing all possible code (i.e., including a lot of irrelevant code) can lead to a reduced grade.

Columns: 4

Sub-Task

Group: Communicatio
Task #1: Communicatio
Sub Task #1: Show

Sub-Task

Group: Communicatio
Task #1: Communicatio
Sub Task #2: Show

Sub-Task

Group: Communicatio
Task #1: Communicatio
Sub Task #3: Show

Sub-Task

Group: Communicatio
Task #1: Communicatio
Sub Task #4: Show

each Client

the code

the code

the code

Task Screenshots

Gallery Style: 2 Columns

4 2 1

Each client sending/receiving a message

Caption(s) (required) ✓

Caption Hint:

Describe/highlight what's being shown

Task Screenshots

Gallery Style: 2 Columns

4 2 1

Gets the user input

Sends the message

Sends payload over the socket

Caption(s) (required) ✓

Caption Hint:

Describe/highlight what's being shown (ucid/date must be present)

Task Response Prompt

Briefly explain the code/logic/flow involved

Response:

The listenToInput() method listens to user input by the client. The method checks if the message is a command when the user types in a message. But if not, it thinks it's a chat message, and calls sendMessage(). With this method, it will create a Payload object with message and then it will invoke the send() method to send it to the server. send() method

Task Screenshots

Gallery Style: 2 Columns

4 2 1

handle received message from the Client

Caption(s) (required) ✓

Caption Hint:

Describe/highlight what's being shown (ucid/date must be present)

Task Response Prompt

Briefly explain the code/logic/flow involved

Response:

The processPayload method receives the different types of payloads from the clients. It checks the type of message and takes the appropriate action: It sets the client's name when a client connects, relays a message when a message is sent to other clients in the same room, manages those actions when a room is created or joined, and removes a client from the room when they disconnect. And it handles errors.

Task Screenshots

Gallery Style: 2 Columns

4 2 1

Listens for messages from the server

formats and displays the messages

Caption(s) (required) ✓

Caption Hint:

Describe/highlight what's being shown (ucid/date must be present)

Task Response Prompt

Briefly explain the code/logic/flow involved

Response:

The listenToServer() method continuously listens for incoming messages. The processMessage() method formats and displays the messages.

writes the Payload to the output of the socket in the direction of sending message to server.

End of Task 1

Task

100%

Group: Communication
Task #2: Rooms
Weight: ~50%
Points: ~1.50

COLLAPSE

Details:

Important: Code screenshots should be fairly concise (try to show only the sections of code relevant to the question)

Capturing all possible code (i.e., including a lot of irrelevant code) can lead to a reduced grade.

Columns: 4

Sub-Task 100% Group: Communicatio Task #2: Rooms Sub Task #1: Show Clients can

Sub-Task 100% Group: Communicatio Task #2: Rooms Sub Task #2: Show Clients can

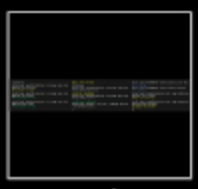
Sub-Task 100% Group: Communicatio Task #2: Rooms Sub Task #3: Show the Client

Sub-Task 100% Group: Communicatio Task #2: Rooms Sub Task #4: Show the

Task Screenshots

Gallery Style: 2 Columns

4 2 1



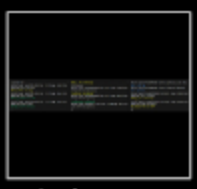
User: Bakr Created a room called House

Caption(s) (required) ✓
Caption Hint:
Describe/highlight what's being shown

Task Screenshots

Gallery Style: 2 Columns

4 2 1



Join/leave message

Caption(s) (required) ✓
Caption Hint:
Describe/highlight what's being shown

Task Screenshots

Gallery Style: 2 Columns

4 2 1

sendJoinRooms

sendCreateRoom()

processClientCommand() that uses create room

Task Screenshots

Gallery Style: 2 Columns

4 2 1

processPayLoad()

handleCreateRoo

handleJoinRoom()

Caption(s) (required) ✓

and join room

Caption(s) (required) ✓
Caption Hint:
Describe/highlight what's being shown (ucid/date must be present)

Task Response Prompt

Briefly explain the code/logic/flow involved
Response:

When a user enters a command of type create room (name) or join room (name) in the client code, it is processed and the content is fed into the processClientCommand method that parses the command and calls the appropriate handler (sendCreateRoom to generate new room or sendJoinRoom to enter a room already exists). These methods construct a Payload object in which type in the Payload is set either to ROOM_CREATE or ROOM_JOIN and set the room name. Then when the payload is sent to the server, the server is initialized to create or join the room. Requests are processed by the server, and the appropriate messages according to user types are broadcast in the room.

Caption Hint:
Describe/highlight what's being shown (ucid/date must be present)

Task Response Prompt

Briefly explain the code/logic/flow involved
Response:

The ServerThread class processes incoming payloads for the code logic of room creation and joining. When a ROOM_CREATE payload is received, it calls handleCreateRoom, which checks if the room can be created, if so, the client is added to the room, otherwise an error message is sent. The handleJoinRoom method checks if the room exists and adds the client if it does, or sends an error if it doesn't for a ROOM_JOIN payload.

Sub-Task 100% Group: Communicatio Task #2: Rooms

Sub-Task 100% Group: Communicatio Task #2: Rooms

Sub Task #5: Show the Server

Sub Task #6: Show that Client

Task Screenshots

Gallery Style: 2 Columns

4 2 1

createRoom() joinRoom()

Caption(s) (required) ✓

Caption Hint:

Describe/highlight what's being shown (ucid/date must be present)

Task Response Prompt

Briefly explain the code/logic/flow involved

Response:

In the Server class, the createRoom method checks if there is a room with the specified name in the rooms map. If it doesn't, it creates a new Room object, puts it in the rooms map, and logs the creation. The first thing the joinRoom method does is verify that the target room exists. If it does, it removes the client from its current room (if any), and retrieves the client's current room. It then adds the client to the specified room, which essentially means the client can switch rooms.

Task Screenshots

Gallery Style: 2 Columns

4 2 1

Client Bakr and User1 see hi. Client3 cant see hi.

Caption(s) (required) ✓

Caption Hint:

Describe/highlight what's being shown

Task Response Prompt

Briefly explain why/how it works this way

Response:

The sendMessage method in the Room class limits client messages to their specific Room. It checks the clientsInRoom list to see if the client is in the same room and only sends the message to other clients in the same room. What this means is that clients in different rooms cannot see or receive each other's messages. ConcurrentHashMap is used to safely manage the clients, so that the messages are delivered correctly without interference from other

clients in different rooms.

End of Task 2

End of Group: Communication
Task Status: 2/2

Group

100%

Group: Disconnecting/Termination
Tasks: 1
Points: 3

COLLAPSE

Task

100%

Group: Disconnecting/Termination
Task #1: Disconnecting
Weight: ~100%
Points: ~3.00

COLLAPSE

Details:

Important: Code screenshots should be fairly concise (try to show only the sections of code relevant to the question)

Capturing all possible code (i.e., including a lot of irrelevant code) can lead to a reduced grade.

Columns: 1

Sub-Task

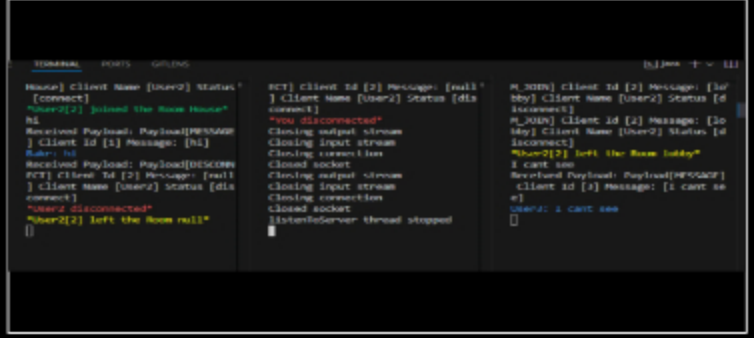
100%

Group: Disconnecting/Termination
Task #1: Disconnecting
Sub Task #1: Show Clients gracefully disconnecting (should not crash Server or other Clients)

Task Screenshots

Gallery Style: 2 Columns

4 2 1



Disconnected without crashing

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Sub-Task 100%

Group: Disconnecting/Termination

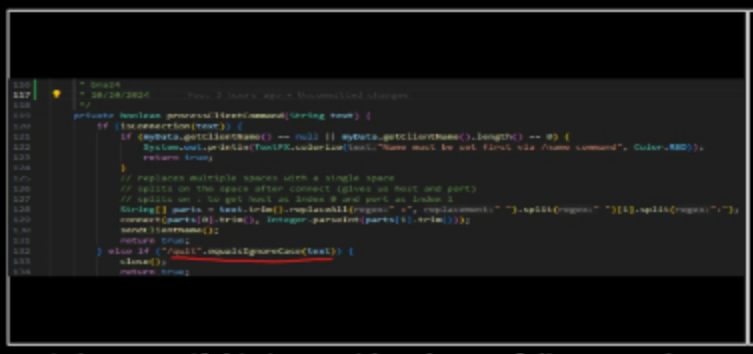
Task #1: Disconnecting

Sub Task #2: Show the code related to Clients disconnecting

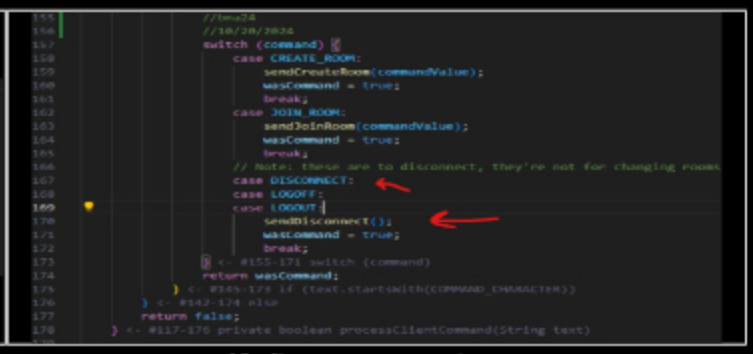
Task Screenshots

Gallery Style: 2 Columns

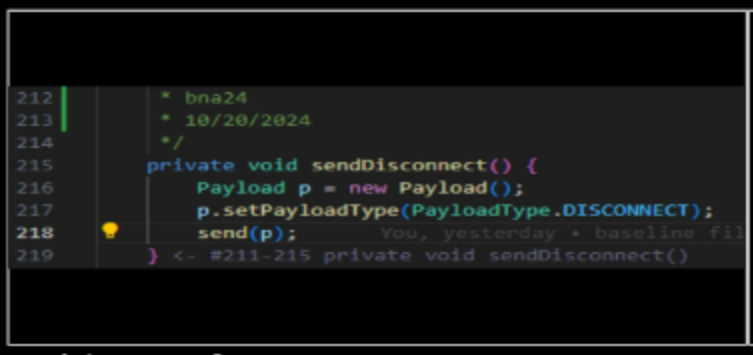
4 2 1



/quit (not sure if this is considered part of disconnect)



processCommand() disconnect portion



sendDisconnect()

processPayload()

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown (ucid/date must be present)*

Task Response Prompt

Briefly explain the code/logic/flow involved

Response:

The client disconnection process is managed through three main methods: sendDisconnect, processClientCommand, and processPayload. When a user sends a disconnect command, processClientCommand sees that and calls sendDisconnect, which creates a Payload object with type DISCONNECT and sends it to the server to indicate that the user wants to disconnect. This message is then processed by the server and when the client receives a DISCONNECT payload via processPayload, the server matches it to the client's ID and executes the disconnection logic.

Sub-Task 100%

Group: Disconnecting/Termination

Task #1: Disconnecting

Sub Task #3: Show the Server terminating (Clients should be disconnected but still running)

Task Screenshots

Gallery Style: 2 Columns

4 2 1

Terminating Server

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Sub-Task

100%

Group: Disconnecting/Termination

Task #1: Disconnecting

Sub Task #4: Show the Server code related to handling termination

Task Screenshots

Gallery Style: 2 Columns

4 2 1

```
//  bna24
// 10/19/2024
private Server(){
    Runtime.getRuntime().addShutdownHook(new Thread(() -> {
        System.out.println(x:"JVM is shutting down. Perform cleanup tasks.");
        shutdown();
    }));
} <- #17-22 private Server()
```

Initiates Shutdown

```
 * bna24
 * 10/21/2024
 */
private void shutdown() {
    try {
        //chose removeIf over forEach to avoid potential ConcurrentModificationException
        //since empty rooms tell the server to remove themselves
        rooms.values().removeIf(room -> {
            room.disconnectAll();
            return true;
        });
    } catch (Exception e) {
        e.printStackTrace();
    }
} <- #56-67 private void shutdown()
```

shutdown()

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown (ucid/date must be present)*

Task Response Prompt

Briefly explain the code/logic/flow involved

Response:

In the constructor, the server handles termination by using a shutdown hook that calls the shutdown method when the JVM is shutting down. The shutdown method iterates through active rooms and removes each room's clients using removeIf to make sure it is safe to remove without causing errors. This design makes sure that all connections are closed properly and resources are released when the server is terminated.

End of Task 1

End of Task 1

End of Group: Disconnecting/Termination
Task Status: 1/1

Group

Group: Misc
Tasks: 3
Points: 1

COLLAPSE

Task

Group: Misc
Task #1: Add the pull request link for this branch
Weight: ~33%
Points: ~0.33

COLLAPSE

Task URLs

URL #1
https://github.com/BakrAbushaar/bna24-
IT114-[illegible]

URL
https://github.com/BakrAbushaar/bna24-IT114-0

End of Task 1

Task

Group: Misc
Task #2: Talk about any issues or learnings during this assignment
Weight: ~33%
Points: ~0.33

COLLAPSE

Details:
Few related sentences about the Project/sockets topics

Task Response Prompt

Response:

Took me a really long time figuring out where everything is and uploading it to this document. This assignment helped me understand how to filter through long code and figure out what code does what.

End of Task 2

Task

100%

Group: Misc
Task #3: WakaTime Screenshot
Weight: ~33%
Points: ~0.33

COLLAPSE

Details:
Grab a snippet showing the approximate time involved that clearly shows your repository.

The duration isn't considered for grading, but there should be some time involved.

Task Screenshots

Gallery Style: 2 Columns

4 2 1

WakaTime

End of Task 3

End of Group: Misc
Task Status: 3/3

End of Assignment